

PATH PLANNING FOR AUTONOMOUS DRONES

Thirisha.R
Department of Mechatronics
Engineering
Hindustan University
Chennai, India

22125022@student.hindustanuniv.ac.in

Shaik Sameer Salahuddin
Department of Mechatronics
Engineering
Hindustan University
Chennai, India

22125004@student.hindustanuniv.ac.in

Dr.S.Janaki Raman
Department of Mechatronics
Engineering
Hindustan University
Chennai,India

janakir@hindustanuniv.ac.in

Dr. N.Sivakamasudhari
Department of Mechatronics
Engineering
Hindustan University
Chennai,India

sksundari@hindustanuniv.ac.in

Abstract—This project compares two popular path-planning algorithms, A*(Among path-planning algorithms) and RRT(Rapidly-exploring Random Trees), for autonomous drone navigation. Simulations in SOFTWARE were used to evaluate their performance in dynamic 3D environments with obstacles, like those in search and rescue or inspection scenarios.

A* systematically finds optimal paths using heuristics, excelling in structured, grid-based environments. In contrast, RRT rapidly explores high-dimensional spaces, prioritizing feasible paths over strict optimality. Key metrics, including path efficiency, computational complexity, and obstacle-handling, revealed that A* is ideal for structured spaces, while RRT suits complex, high-dimensional scenarios.

The study provides insights into choosing the right algorithm for specific autonomous navigation tasks.

Keywords-A*,RRT,pathplanning,simulations,nodes,navigation

I. INTRODUCTION

Autonomous drones have revolutionized industries like logistics, agriculture, surveillance, and disaster response by addressing challenges in complex or hazardous environments. Effective path planning is critical to their success, enabling safe navigation by creating obstacle-free routes and adapting to environmental changes. Among path-planning algorithms, A* and Rapidly-exploring Random Trees (RRT) are prominent. A* excels in structured environments, using heuristic functions to efficiently identify optimal paths with low computational demands, making it ideal for real-time navigation. RRT, on the other hand, is well-suited for high-dimensional, cluttered spaces. Its probabilistic approach rapidly generates feasible paths, making it effective in dynamic scenarios like search and rescue. This study compares A* and RRT using SOFTWARE simulations, evaluating computational efficiency, path quality, and adaptability. By analyzing their performance across diverse scenarios, it provides insights into selecting the most suitable algorithm for specific tasks, advancing the development of reliable and efficient autonomous navigation systems.

Recent developments in UAV Path Planning have illustrated a shift away from the traditional used algorithms like

Dijkstra's and A* to adaptive and hybrid techniques. Dijkstra's algorithm guarantees paths are optimal but has a high cost in terms of computation time making it impractical for applications that need real-time results [2]. The A* algorithm improves upon Dijkstra's algorithm and is able to produce paths in real-time for more massive maps using heuristic search [4]. Studies that perfectly address dynamic navigation challenges have developed 3D Dubins curves integrated with RRT to provide smooth trajectories, while also rapidly re-planning paths when the environment changes [1]. Other studies include a modified Dijkstra-A* algorithm that optimizes multi-UAV operations using distance, battery life, and area coverage [3]. Additionally, RRT and other sampling-based methods have demonstrated timely performance past motion planning for UAVs, as the modified RRT (MRRT) algorithm came about to improve non-collision trajectories along with real-time planning capabilities [5].

Other comprehensive reviews categorize UAV path planning problems into three different types: traditional approaches, intelligent approaches, and fusion approaches. The fusion methods are an attempt to provide both efficiency and adaptively for dynamic and changing conditions [6], [7]. Collaborative planning techniques that use formations of UAVs have also been considered to improve coordination in multi-UAV environments [8], and massive literature reviews have noted a trend for improved environmental representation and feasibility of robust path generation in UAV path planning [9].

There have also been innovative strategies proposed such as spiking neural networks (SNN) [10], reinforcement learning-based APPA-3D with collision-safety envelopes [11], and hybrid methods that couple A* with Dynamic Window Approach (DWA) to provide smooth adaptations in real-time [12]. In recent works, scalability and responsiveness has been studied using an AI system processed in a cloud [13]. However, an important gap in this literature has not addressed the combination that allows the optimality and precision of A* with the fast adaptability of RRT to provide a responsive, scalable, and collision-free navigation in challenging 3D dynamic environments with moving obstacles.

II. METHODOLOGY

This research begins with a literature review of UAV path-planning algorithms, analyzing their adaptability, efficiency, collision avoidance, and path smoothness in static and dynamic environments. Algorithms like 3D Dubins Curve with RRT, Dijkstra's, A*, and MRRT will be compared, and findings summarized in a table to select the two most suitable for further analysis. The study uses SOFTWARE for simulations and animations.

A. Types Of Path Planning Algorithms

From the [6] we get to know the types of algorithms and their classification.

Traditional algorithms like Dijkstra's, A*, and RRT are more efficient than machine learning-based algorithms for UAV path planning, particularly in stable environments. Traditional algorithms, based on deterministic principles and heuristic-guided searches, deliver reliable pathfinding with lower computational demand, making them well-suited for real-time applications in semi-static settings. machine learning methods, such as reinforcement learning and spiking neural networks, provide adaptability and advanced decision-making capabilities in highly dynamic environments but at the cost of increased computational overhead. Thus, while machine learning algorithms excel in complex, unpredictable scenarios, traditional algorithms are generally more efficient and resource-friendly for simpler, more predictable UAV path-planning tasks.

After choosing a traditional algorithm and comparing Dijkstra, A* and RRT for a comparison from [13],[12],[5].

Python implementations of path-planning algorithms based on methods from studies [5], [12], and [13]. A* and RRT are compared for their unique strengths in UAV path planning: A* excels in finding near-optimal paths with efficient heuristics, ideal for real-time applications, while RRT rapidly explores high-dimensional spaces, navigating complex, dynamic obstacles. Their complementary features offer valuable insights into algorithm performance under varying conditions.

B. Algorithm selection

The A* algorithm is optimal for structured environments, balancing search efficiency and path accuracy, making it ideal when precision is critical. It employs a heuristic-based cost function ($f(n)=g(n)+h(n)$) to evaluate nodes, selecting the one with the lowest cost for efficient pathfinding. Euclidean distance serves as the heuristic due to its relevance in 3D environments and its ability to approximate distances accurately. In contrast, the RRT algorithm is well-suited for complex, dynamic environments where rapid, adaptable path generation is required, prioritizing obstacle avoidance over optimality. RRT grows a tree by iteratively

connecting random points to the nearest tree node, ensuring collision-free paths, with a focus on extensive exploration rather than the shortest path.

C. Environmental setup

Now after selection of algorithm, we select the parameters such as environment setup including start and end points, and obstacles. For environmental representation [9].

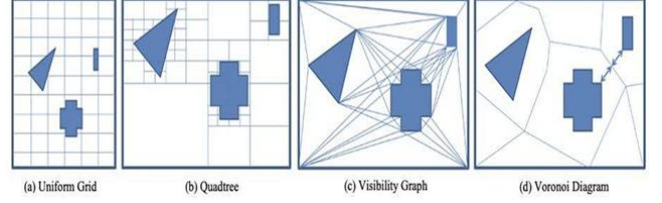


Figure 1 different type of environment

The environment is divided into a uniform grid (Figure 1) of fixed-size cells or voxels, simplifying path computation. Cells with obstacles are marked as occluded, while obstacle-free cells are labelled as free. Though this method guarantees a solution within finite time, it is memory-intensive and unsuitable for real-time tasks. The Uniform Grid method is chosen for its simplicity and ease of implementation.

The 3D occupancy grid is a volumetric representation of the environment, marking each cell as either occupied (obstacle) or unoccupied (free space), enabling obstacle detection and path evaluation. Path-planning algorithms like A* and RRT* were integrated and simulated within this grid, visualized in SOFTWARE to represent start and goal points, obstacles, and algorithm-generated paths. The grid, modeled as a [20, 20, 10] structure, provided a balance between computational efficiency and navigational precision, with each cell representing a specific coordinate for path exploration. Grid boundaries simulated realistic spatial constraints such as walls and structural limits, while obstacles were strategically placed to mimic real-world barriers, including vertical walls, clustered objects, and isolated obstructions. SOFTWARE's flexibility allowed for dynamic adjustments to grid parameters, obstacle layouts, and densities, enabling repeated testing under diverse configurations. Start and goal positions were strategically placed at grid extremities or near obstacles to test algorithm efficiency and adaptability in navigating long, challenging paths or dynamically changing targets. This setup created a realistic, controlled environment to assess the performance and robustness of A* and RRT algorithms in complex 3D navigation tasks.

The algorithm implementation is based on a comprehensive literature review, drawing on research from papers such as [3], [5], [12], and [13].

A* algorithm

A* builds upon Dijkstra's algorithm by incorporating a heuristic function to guide the search more efficiently. It reliably finds the shortest path from the start to the goal,

ensuring completeness and optimality. Utilizing two sets—Open and Closed—A* iteratively expands the node with the lowest cost. Path planning concludes when the goal node is added to the Closed set or when the Open set is empty. The node cost is calculated by

Figure 2,[12] node calculation

$$f(n) = g(n) + h(n) \dots\dots\dots(1)$$

In Figure 6, (f(n)) represents the total estimated cost from the starting point, through node (n), to the target point. Here, (g(n)) denotes the actual cost from the starting point to node (n), while (h(n)) is the heuristic estimated cost from node (n) to the target. The calculation of (h(n)) typically uses one of two methods: Euclidean Distance or Manhattan Distance, as specified below.

Figure 3,[12] Euclidean distance and Manhattan distance

$$h(n) = \sqrt{(m_x - n_x)^2 + (m_y - n_y)^2} \dots\dots\dots(2)$$

$$h(n) = |m_x - n_x| + |m_y - n_y| \dots\dots\dots(3)$$

In Figure 4, (mx, my) represents the coordinates of the current node, while (nx, ny) represents the coordinates of the target node. Given the complexity and computational intensity of calculating the Euclidean Distance, this paper adopts the Manhattan Distance formula. While the A* algorithm can identify the shortest path needed to complete a task in a known environment, it has a wide search range, lower planning efficiency, and cannot avoid unknown obstacles. [12].

RRT algorithm

The main objective is to create a collision-free path linking the specified start point ((x_s, y_s, z_s)) to the goal point ((x_g, y_g, z_g)) within the workspace. A collision-free path ensures that, at any moment, no static or dynamic obstacles ((x_{O_i}(t))) enter the UAV's safety zone (O(x(t))). This path is formulated as follows:

Figure 4 ,[5] path of RRT

$$f(x, y, z) = 0 \dots\dots\dots(4)$$

Here, O_i) represents each obstacle, with its position predicted using a constant velocity model at time (t). The anticipated position of the obstacles is given by:

Figure 5,[5] predicted position of the obstacles

$$x_{oi}(t) = x_{oi}(t_o) + v_{oi}(t_o) * (t - t_o) \dots\dots\dots(5)$$

The shortest path length is determined by integrating the differential arc length along the defined trajectories.

Figure 6,[6] Differential arc length along the paths

$$f^* = \arg \min_f \int_f ds \dots\dots\dots(6)$$

The assumptions are as follows: (a) obstacles move with a constant velocity, and (b) the path planner developed using the algorithm has access to the known positions and

velocities of the obstacles, as well as the states of the UAV. The program code is implemented based on equations (1, 2, 3, 4, and 5), and the workflow of the program is discussed

III. ALGORITHM IMPEMENTATION IN SOFTWARE

A*Algorithm Implementation,Structured nodes with position, parent, and cost fields (G, H, F). Open and Closed lists managed node evaluation, and the shortest path was reconstructed by tracing parent nodes upon reaching the goal. A 3D plot visualized obstacles, the explored path, and the final route.

RRTAlgorithm Implementation,The tree expanded from the start node toward random points, connecting to the nearest tree node if obstacle-free. The path was traced back through parent nodes upon reaching the goal. A 3D visualization depicted tree growth, new edges, and the final path.

Performance Metrics

Efficiency: Measured computational time and memory usage. A* required more memory for Open/Closed lists, while RRT was faster with minimal storage.

Path Optimality: Compared path distances and obstacle avoidance. A* generated shorter, precise paths; RRT paths were longer due to random exploration.

Scalability: Tested varying grid sizes. A* faced higher computational demand in larger grids, while RRT maintained speed with sparse node checking.

A. Comparative Anaylsis Based On Observation

Algorithm Suitability for Autonomous Navigation:

- A in Structured Environments*: Found to be ideal for environments with clear structure and defined boundaries. A* reliably produced shorter, more accurate paths, beneficial for navigation in tightly controlled spaces.
- RRT in Dynamic or Unstructured Spaces: RRT showed superior performance in quickly identifying feasible paths in complex environments. Although its paths were suboptimal, its speed and adaptability make it preferable in scenarios where real-time adjustments are required.

Hybrid Approach Consideration: Observed that a combination of both methods could enhance overall path-planning quality, where RRT quickly finds a path, and A* refines it for optimality.

B. Simulation and Results Visualistion

• 3D Path Visualization: Presented both algorithms' paths in 3D simulations, showing the step-by-step progress of each algorithm and the final path with clear markers for start, goal, and obstacles.

• Performance Metrics Display: Displayed computational time and path length for each algorithm in each test scenario

to provide visual and quantitative data on their respective efficiencies and optimality.

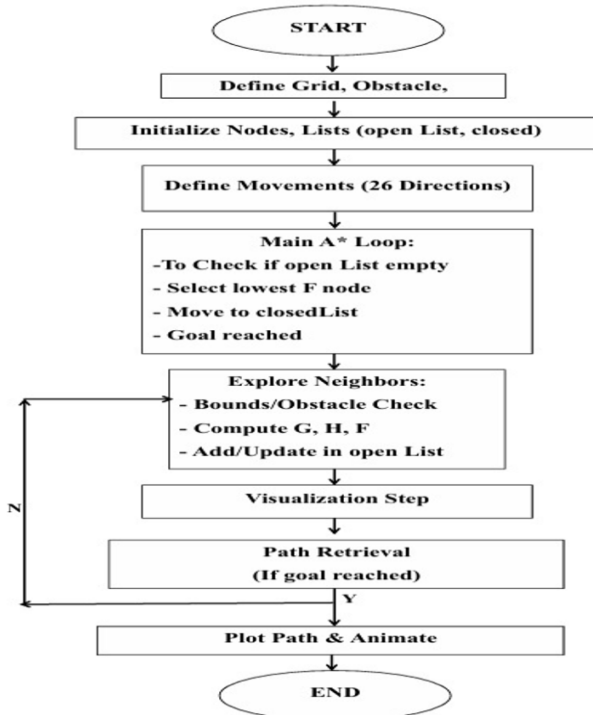
- **Dynamic Animation:** Added animation sequences for both A* and RRT path-following, allowing visualization of a virtual drone's progress along the planned route.

- **Performance and Path Quality:**

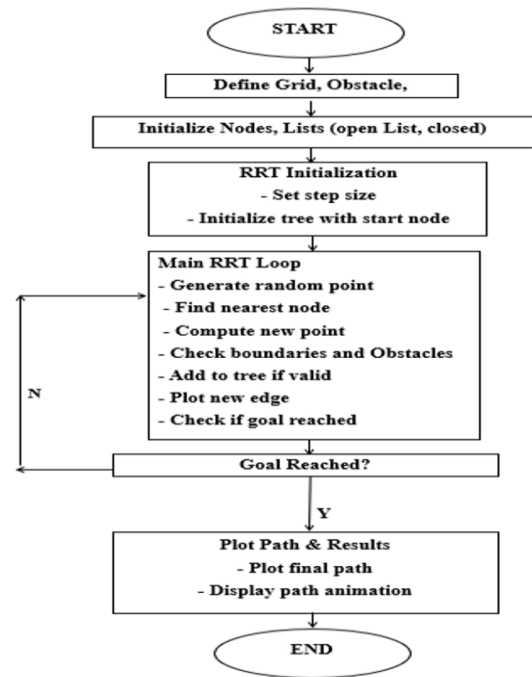
A* consistently generated optimal paths but required more computation time and memory, especially in larger grids. RRT offered faster path generation but with longer paths, which may be acceptable in environments prioritizing collision-free paths over optimal length.

C. Code Flow chart

A*Algorithm



RRT Algorithm



V RESULT

A. Simulation Result

Both A* and RRT were able to find paths from the start to the goal in various test environments with obstacles. The results showed clear differences in terms of path quality and computational performance.

B. A*Algorithm Result

- A* consistently found the optimal path in environments with structured, grid-based layouts.

- It demonstrated higher path smoothness and optimality, avoiding unnecessary detours.

- However, the algorithm was slower, especially in complex environments, due to the need for exhaustive node expansion.

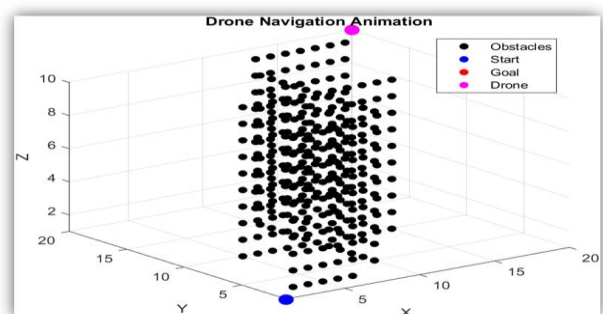


Figure 11 : Drone Navigation

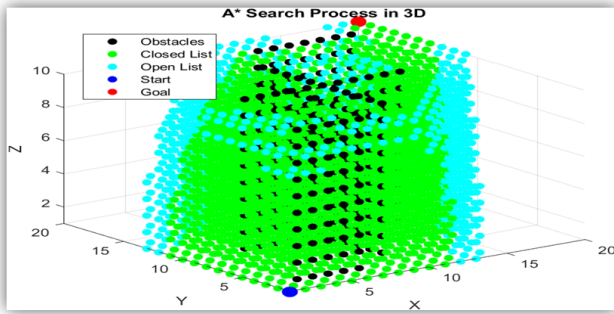
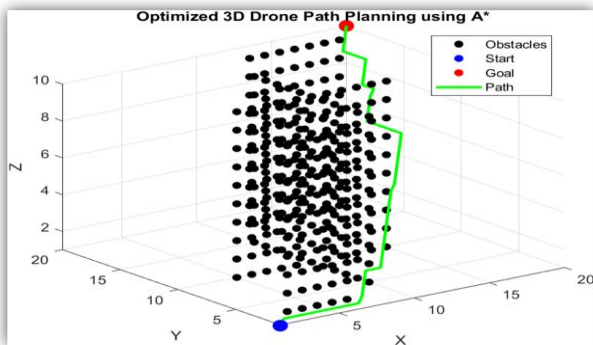


Figure 22 : A*searching Process

The Above Figure 11 Depicts the Drone Navigation Animation Where Drone first Navigates the Obstacles.

The Figure 12 Depicts the Search Process of A* Algorithm in 3D and finds a Path.

Figure 13:Optimized 3D Drone Path Planning using A*



The Figure 13 Depicts the optimized 3D path that is used to reach the Goal From Start point

C.RRT Algorithm Result

- RRT showed faster performance in most test scenarios due to its sampling-based approach.
- While the paths were feasible, they were not always optimal and often involved jagged movements due to random sampling.
- RRT was better suited for complex spaces where traditional grid-based approaches are less efficient.

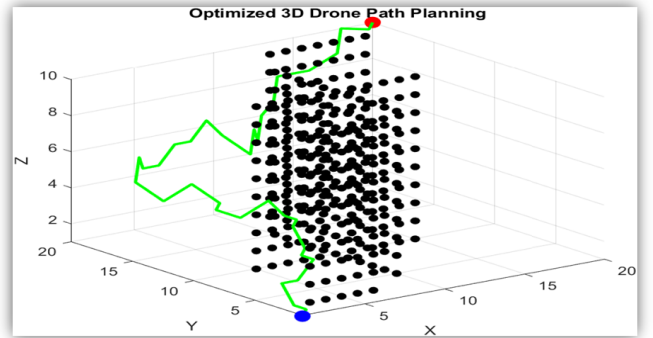
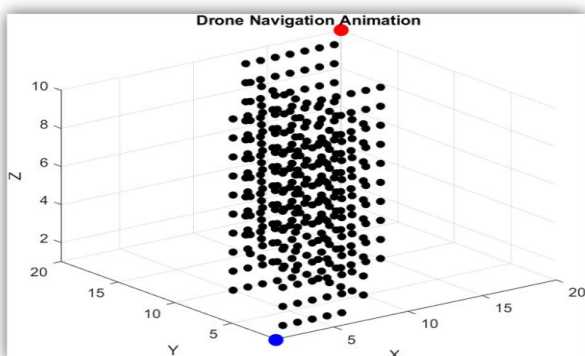


Figure 14: Drone navigation

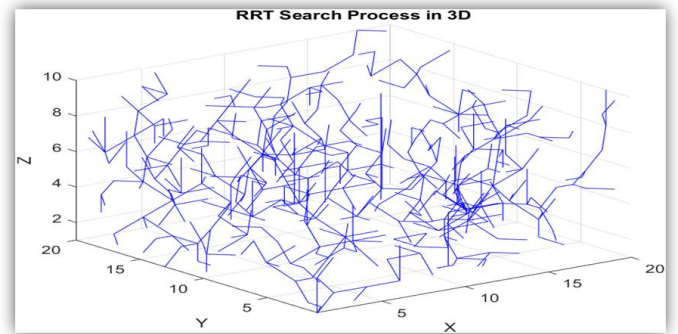


Figure 15 RRT search process in 3D

Figure 16 optimized 3D drone path planning

The Above Figure 14 Depicts the Drone Navigation Animation Where Drone first Navigates the Obstacles.

The Figure 15 Depicts the Search Process of RRT Algorithm in 3D and finds a Path.

The Figure 16 Depicts the optimized 3D path that is used to reach the Goal From Start point.

C. Comparative analysis

Comprehensive comparison of A* and RRT algorithms, highlighting their distinct features and suitability for autonomous drone navigation. A* excels in generating optimal paths by systematically evaluating nodes using a cost function $f(n)=g(n)+h(n)$, making it ideal for structured environments where precision is critical. Its heuristic-driven approach ensures efficiency in scenarios with well-defined constraints, such as indoor navigation or grid-based maps. However, A* is computationally intensive, as it requires maintaining Open and Closed lists to evaluate and store all possible nodes, leading to higher memory usage, particularly in large or high-resolution grids. In contrast, RRT focuses on rapid exploration and adaptability, making it well-suited for unstructured, complex, and dynamic environments. Its random sampling strategy enables it to navigate high-dimensional spaces effectively, even in the presence of moving obstacles or unpredictable conditions, such as in search and rescue operations. While RRT does not guarantee optimal paths and often yields longer routes

due to its exploration-first approach, it is computationally lighter and faster since it does not store all intermediate nodes and focuses on incremental tree expansion. Additionally, A* is easier to implement due to its systematic methodology, while RRT is more complex but offers better handling of dynamic obstacles. Overall, the comparison underscores A* as a precision-focused algorithm for structured scenarios and RRT as a flexible, faster solution for navigating complex and dynamic environments.

VI CONCLUSION

This report analyzes A* and RRT algorithms for autonomous drone path planning, focusing on search and rescue operations. A* excels in structured environments, using heuristics to calculate optimal paths efficiently and avoid obstacles, as demonstrated in SOFTWARE simulations. Its precision and real-time adaptability make it robust for predictable scenarios.

RRT, in contrast, prioritizes exploration over optimization, navigating complex, dynamic environments effectively.

.While its paths are less optimal, RRT's adaptability suits chaotic, unpredictable scenarios.

The project emphasizes hardware integration and functional testing, bridging simulations and real-world applications. Insights from this study enhance autonomous drone systems, contributing to more efficient and effective search and rescue operations while laying a foundation for future advancements

ACKNOWLEDGMENT

First, we thank the Lord Almighty for His blessings throughout this project. We are grateful to the HITS management for their support and facilities. Special thanks to the **Vice-Chancellor, Dr. S. N. Sridhara**, for this opportunity, and to **Dr. S. Janaki Raman, Associate Professor and HoD i/c**, Department of Mechatronics Engineering, for his guidance and encouragement. We appreciate our guide, **Dr. S. Janaki Raman**, for his mentorship and valuable suggestions. Thanks also to our project coordinators, **Dr. S. Janaki Raman and Sivakamasudari**, for their steady supervision, and to all the technical and teaching staff of the department. Finally,

heartfelt thanks to my project partner and to my parents for their unwavering support.

REFERENCES

1. Yucong Lin And Srikanth Saripalli, Path Planning Using 3D Dubins Curve For Unmanned Aerial Vehicles, 2016. [1]
2. Kevin Dananicer, Delphine Ruvio, Inkyung Sung, Peter Neislen, Comparison Of Path Planning Algorithms For An Unmanned Aerial Vehicle Deployment Under Threats, 2019. [2]
3. Dhruv Karve, Farhan Kapadia, Multi-UAV Path Planning Using Modified Dijkstra's Algorithm, 2020. [3]
4. Dian Rachmawati, Lysander Gustin, Analysis Of Dijkstra's Algorithm And A* algorithm In Shortest Path Problem, 2020. [4]
5. A Saravanakumar, A Kaviyarasu And R Ashly Jasmine, Sampling Based Path Planning Algorithm For UAV Collision Avoidance, 2021. [5]
6. Hao Wang And Weijun Pan, Research On UAV Path Planning Algorithms, 2021. [6]
7. Vemema Kangundel · Rodrigo S. Jamisola Jr.1 · Emmanuel K. Theophilus1, A Review On Drones Controlled In Real-Time, 2021. [7]
8. Yunhong Yang, Xingzhong Xiong And Yuehao Yan, UAV Formation Trajectory Planning Algorithms: A Review, 2023. [8]
9. Gopi Gunganand Anwar Haque, Path Planning for Autonomous Drones: Challenges and Future Directions, 2023. [9]
10. Jintao Wang^{1,2}, Zuyi Zhao¹, Jiayi Qu¹ & Xingguo Chen¹, APPA 3D: An Autonomous 3D Path Planning Algorithm For Uavs In Unknown Complex Environments, 2024. [10]
11. Yong He, Ticheng Hou & Mingran Wang, Brain-Inspired Path Planning Algorithms For Drones, 2024. [11]
12. Adinda Putri Sahrita¹, Rafiantika Megahnia Prihandini¹, Alvian Bagus Agatha¹, Graph Theory: Application Of Graphs To The Shortest Path Problem With Dijkstra's Algorithm, 2024. [12]
13. Yixun Chao^{1,2}, Philipp Augenstein², Arne Roennau², Ruediger Dillmann² And Zhi Xiong¹, Brain Inspired Path Planning Algorithms For Drones, 2023. [13]